Algorithm: Email Pattern Finder Using Regular Expressions

1. Start the program.
2. Import the **re** module for regular expression operations.
3. Define the email pattern using a regular expression.
4. Create the **find_emails(text)** function to search the complete text.
5. Use **re.findall()** to collect all substrings matching the email pattern.
6. Return the list of found email addresses.
7. Create the **is_valid_email(candidate)** function to validate a single email address.
8. Use **re.fullmatch()** so that the complete candidate string must match the pattern.
9. Return True if the candidate is a valid email; otherwise return False.
10. Start the demo/driver code.
11. Store a sample block of text containing valid and invalid email-like strings.
12. Display the original text.
13. Call **find_emails(sample_text)** to find all email addresses.
14. Display the number of email addresses found.
15. Display each email address found in the text.
16. Create test cases containing valid and invalid email strings.
17. Validate each test case using **is_valid_email()**.
18. Display VALID for matching email addresses and INVALID for non-matching strings.
19. End the program.

Regular Expression Implementation - Python Source Code

```
"""
==================================================
==================
 EMAIL PATTERN FINDER USING REGULAR EXPRESSIONS
==================================================
==================
IN PLAIN ENGLISH:
  A regular expression (regex) is a SEARCH
TEMPLATE. Instead of
  looking for one exact word, we describe the SHAPE
of what we want:
  "some characters, then @, then more characters,
then a dot, then a
  few more characters." Python then scans the text
and pulls out
  every piece that matches that shape \u2014 in
this case, email addresses.
==================================================
==================
"""

import re


#
==================================================
==================
# 1. THE EMAIL PATTERN
#    Plain English: local part + @ + domain name
+ . + domain suffix
#
==================================================
==================
EMAIL_PATTERN =
r'[a-zA-Z0-9_.+-]+@[a-zA-Z0-9-]+\.[a-zA-Z0-9.-]+'


#
==================================================
==================
# 2. FIND ALL EMAILS IN A BLOCK OF TEXT
#    Plain English: scan the whole text and collect
every substring
#    that matches the email shape, wherever it
```

Regular Expression Implementation - Python Source Code

```
appears.
#
==========================================================
==================
def find_emails(text):
    """Returns a list of every email-shaped
substring found in `text`."""
    return re.findall(EMAIL_PATTERN, text)


#
==========================================================
==================
# 3. VALIDATE A SINGLE STRING AS AN EMAIL
#    Plain English: fullmatch requires the ENTIRE
string to match the
#    pattern from start to end \u2014 useful for
validating ONE address
#    typed into a form, rather than searching
inside a paragraph.
#
==========================================================
==================
def is_valid_email(candidate):
    """Returns True only if the ENTIRE string is a
valid email address."""
    return re.fullmatch(EMAIL_PATTERN, candidate)
is not None


#
==========================================================
==================
# 4. DEMO / DRIVER CODE
#
==========================================================
==================
if __name__ == "__main__":

    sample_text = """
    Please contact us for support:
    - General queries: support@examplecorp.com
    - Sales team: sales.team@business-hub.co.in
```

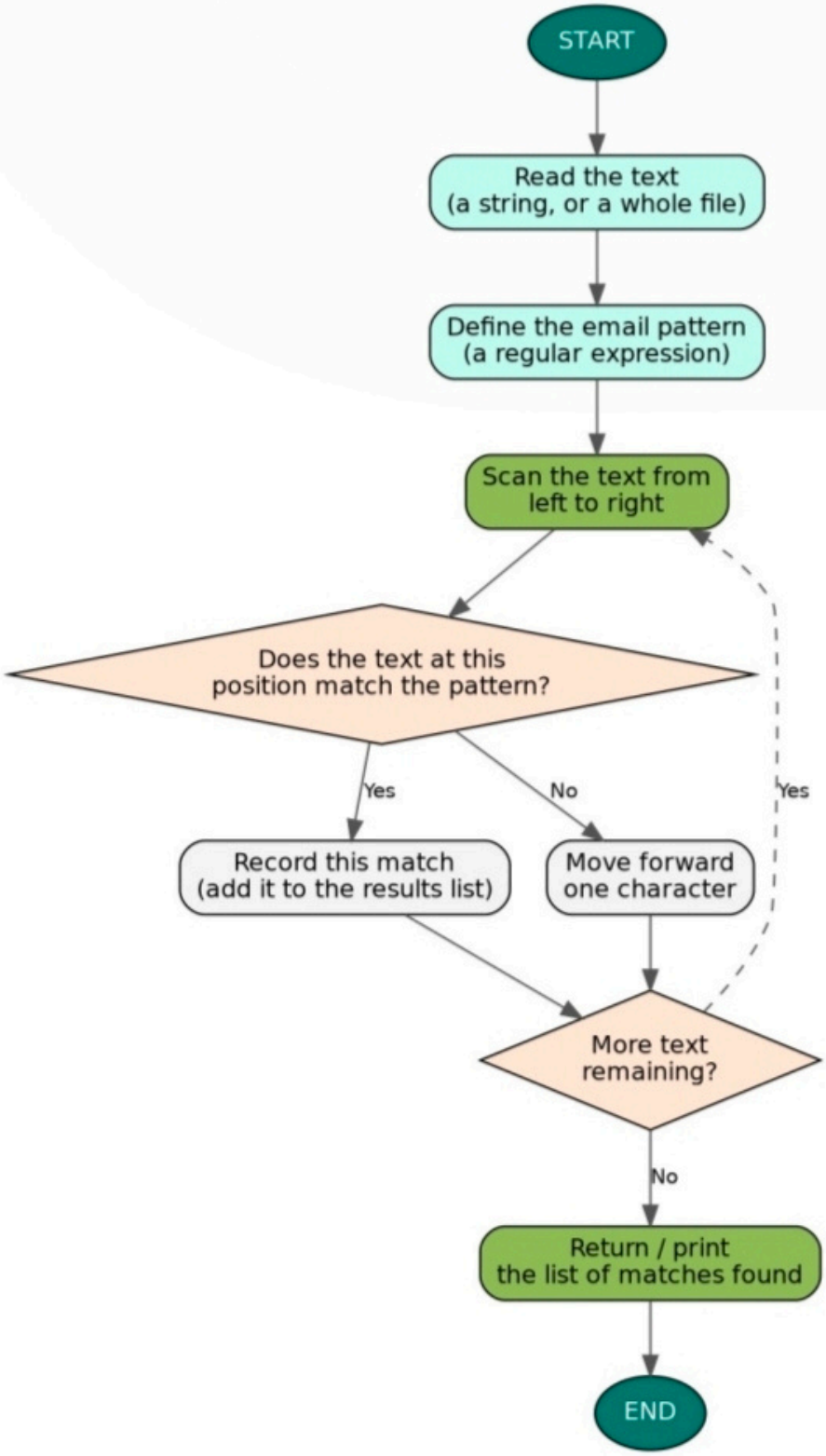
Regular Expression Implementation - Python Source Code

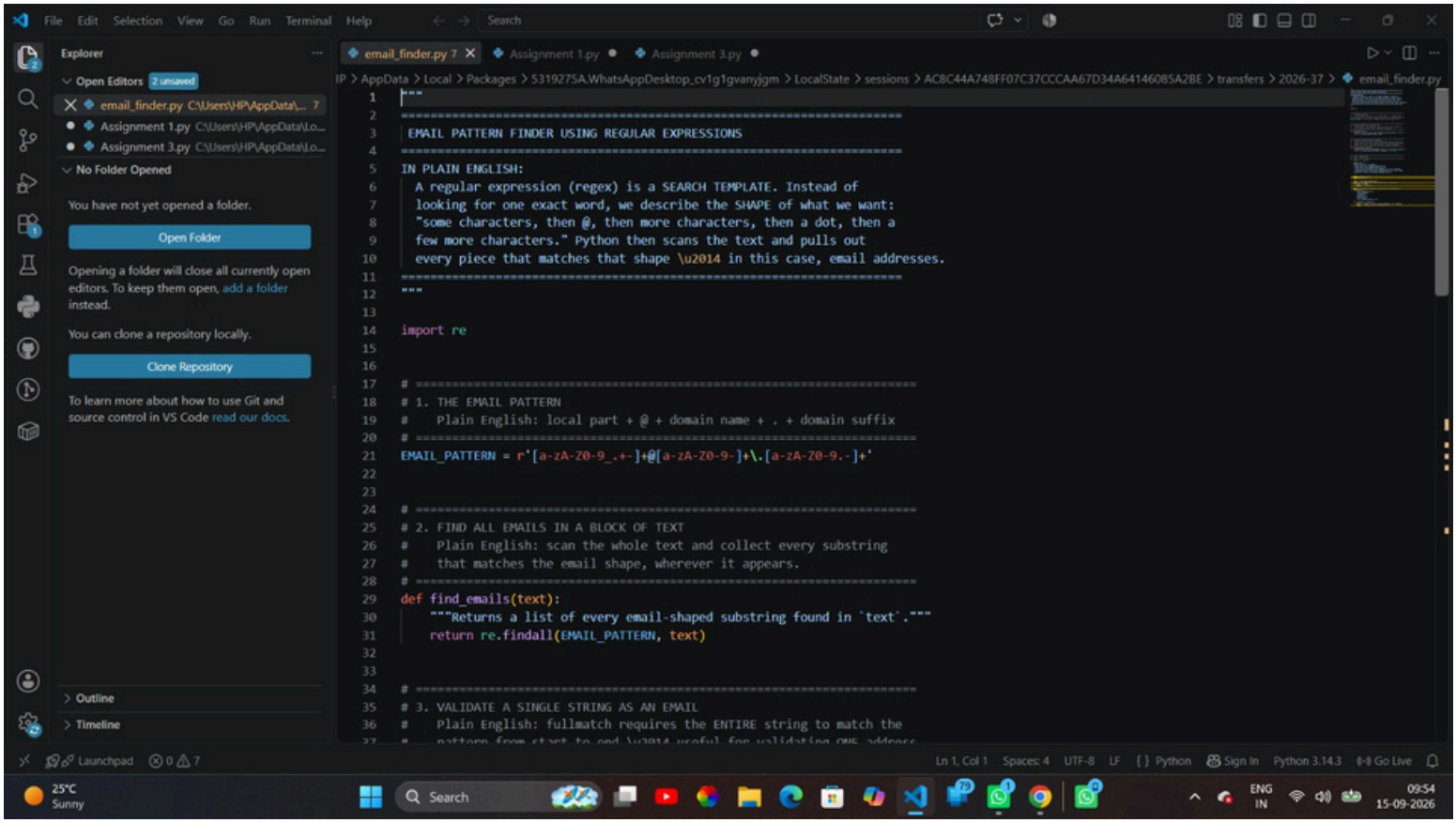
```
    - Personal note from Rahul (rahul_23@gmail.com)
sent yesterday.
    - Invalid mentions: not-an-email,
@missing-local.com, plain.text@
    - Newsletter sign-up:
newsletter+promo@my-site.org
    """

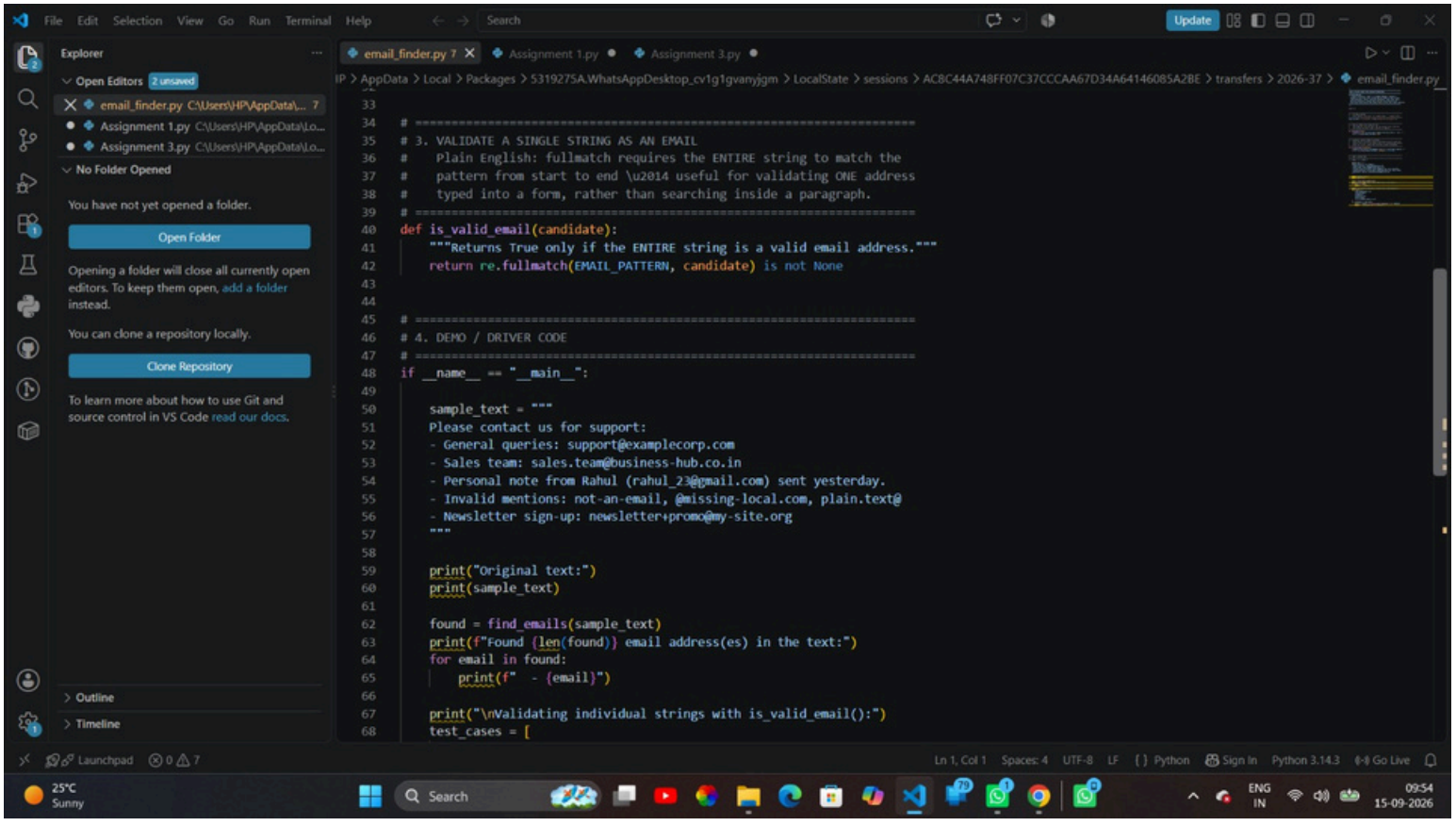
    print("Original text:")
    print(sample_text)

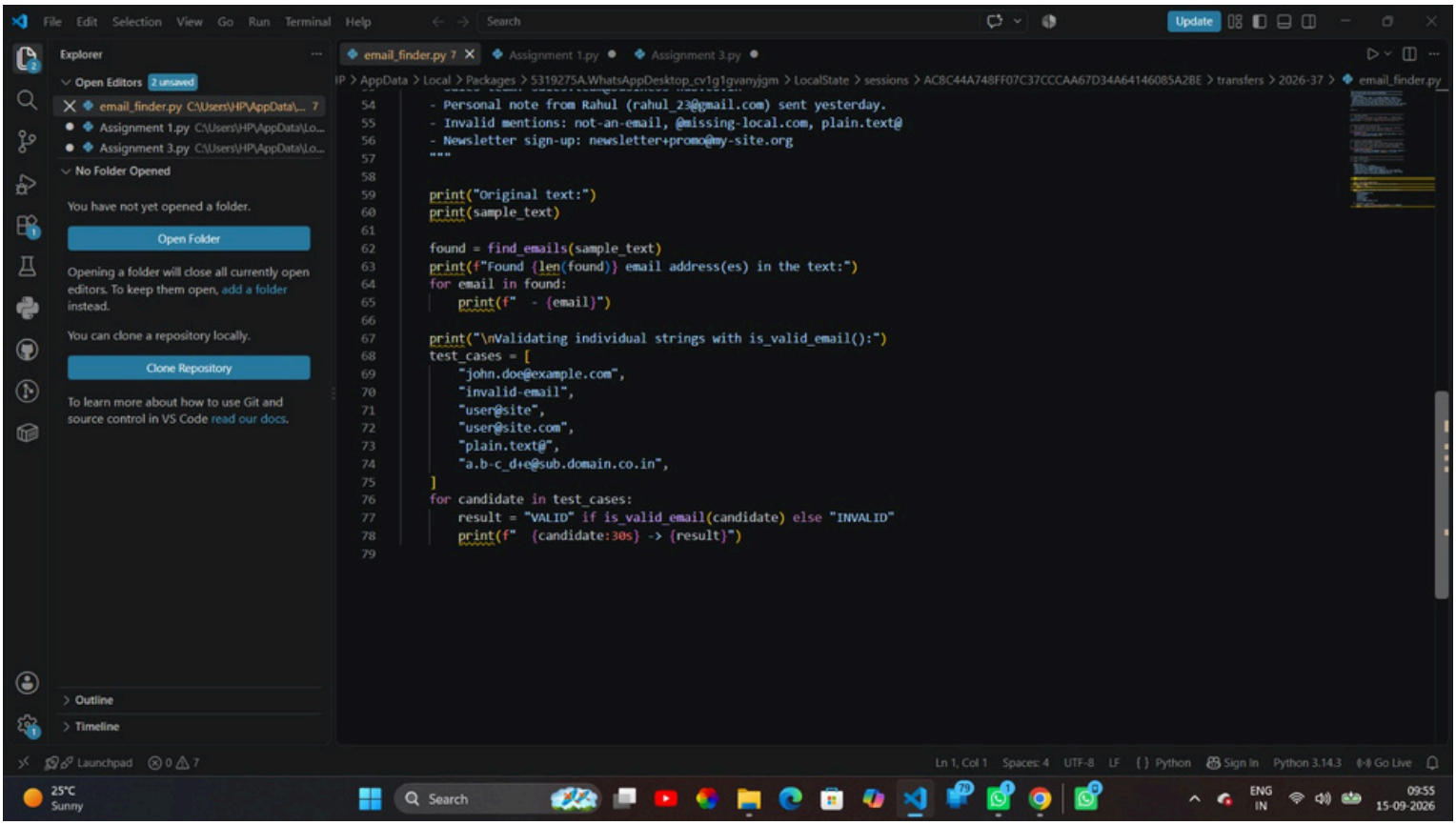
    found = find_emails(sample_text)
    print(f"Found {len(found)} email address(es) in
the text:")
    for email in found:
        print(f"  - {email}")

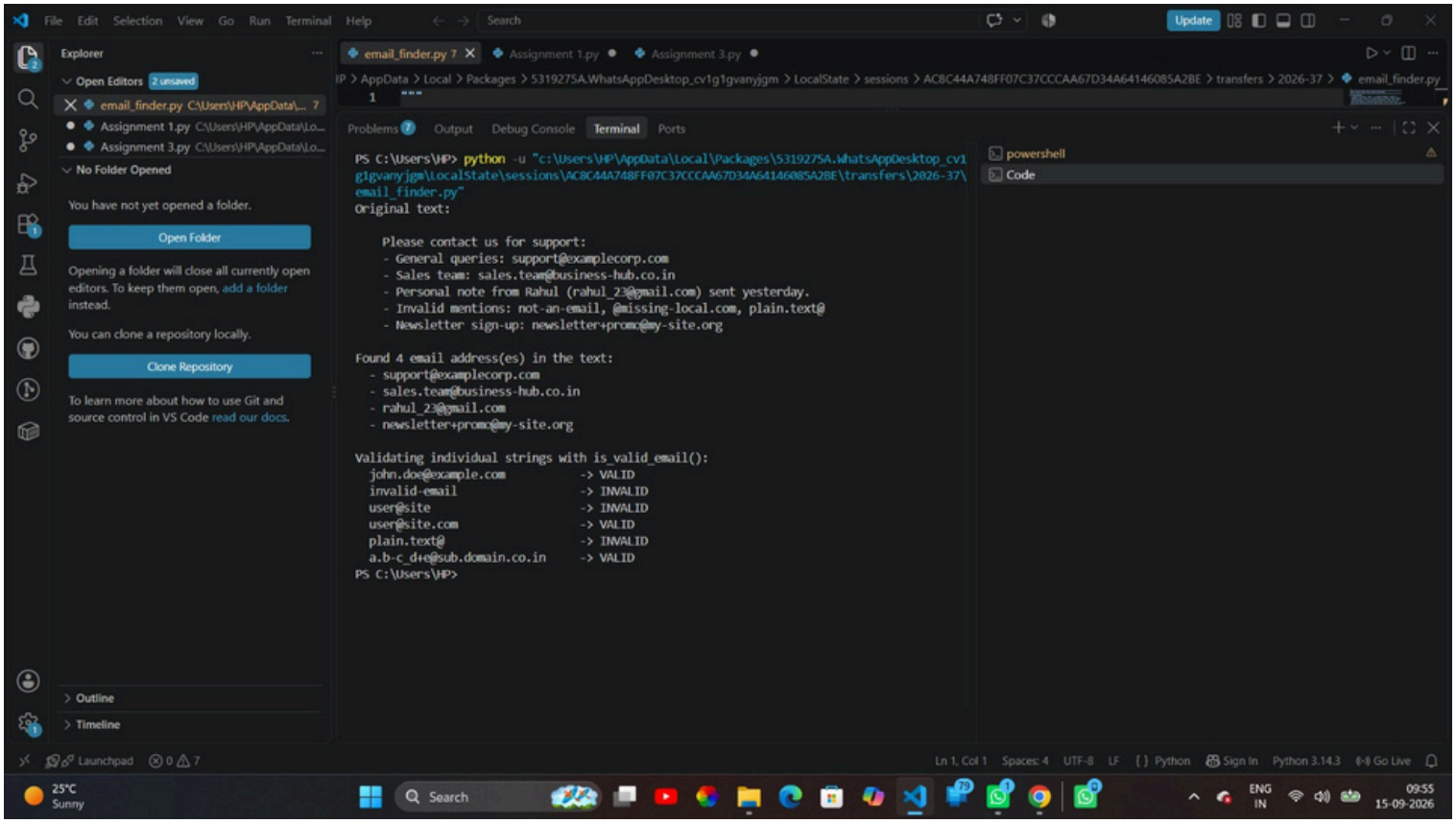
    print("\nValidating individual strings with
is_valid_email():")
    test_cases = [
        "john.doe@example.com",
        "invalid-email",
        "user@site",
        "user@site.com",
        "plain.text@",
        "a.b-c_d+e@sub.domain.co.in",
    ]
    for candidate in test_cases:
        result = "VALID" if
is_valid_email(candidate) else "INVALID"
        print(f"  {candidate:30s} -> {result}")
```


START
Read the text
(a string, or a whole file)
Define the email pattern
(a regular expression)
Scan the text from
left to right
Does the text at this
position match the pattern?
Yes
No
Yes
Record this match
(add it to the results list)
Move forward
one character
More text
remaining?
No
Return / print
the list of matches found
END


email_finder.py 7
Assignment 1.py
Assignment 3.py
Explorer
Open Editors 2 unsaved
No Folder Opened
You have not yet opened a folder.
Open Folder
Clone Repository
"""
EMAIL PATTERN FINDER USING REGULAR EXPRESSIONS
IN PLAIN ENGLISH:
A regular expression (regex) is a SEARCH TEMPLATE. Instead of
looking for one exact word, we describe the SHAPE of what we want:
"some characters, then @, then more characters, then a dot, then a
few more characters." Python then scans the text and pulls out
every piece that matches that shape \u2014 in this case, email addresses.
import re
EMAIL_PATTERN = r'[a-zA-Z0-9_.+-]+@[a-zA-Z0-9-]+\.[a-zA-Z0-9.-]+'
def find_emails(text):
"""Returns a list of every email-shaped substring found in `text`."""
return re.findall(EMAIL_PATTERN, text)
3. VALIDATE A SINGLE STRING AS AN EMAIL
Plain English: fullmatch requires the ENTIRE string to match the


email_finder.py 7
Assignment 1.py
Assignment 3.py
Explorer
Open Editors 2 unsaved
No Folder Opened
You have not yet opened a folder.
Open Folder
Clone Repository
3. VALIDATE A SINGLE STRING AS AN EMAIL
Plain English: fullmatch requires the ENTIRE string to match the
pattern from start to end \u2014 useful for validating ONE address
typed into a form, rather than searching inside a paragraph.
def is_valid_email(candidate):
"""Returns True only if the ENTIRE string is a valid email address."""
return re.fullmatch(EMAIL_PATTERN, candidate) is not None
4. DEMO / DRIVER CODE
if __name__ == "__main__":
sample_text = """
Please contact us for support:
- General queries: support@examplecorp.com
- Sales team: sales.team@business-hub.co.in
- Personal note from Rahul (rahul_23@gmail.com) sent yesterday.
- Invalid mentions: not-an-email, @missing-local.com, plain.text@
- Newsletter sign-up: newsletter+promo@my-site.org
"""
print("Original text:")
print(sample_text)
found = find_emails(sample_text)
print(f"Found {len(found)} email address(es) in the text:")
for email in found:
print(f" - {email}")
print("\nValidating individual strings with is_valid_email():")
test_cases = [


File Edit Selection View Go Run Terminal Help
Search
Update
Explorer
Open Editors 2 unsaved
email_finder.py C:\Users\HP\AppData\... 7
Assignment 1.py C:\Users\HP\AppData\Lo...
Assignment 3.py C:\Users\HP\AppData\Lo...
No Folder Opened
You have not yet opened a folder.
Open Folder
Opening a folder will close all currently open editors. To keep them open, add a folder instead.
You can clone a repository locally.
Clone Repository
To learn more about how to use Git and source control in VS Code read our docs.
Outline
Timeline
email_finder.py 7
Assignment 1.py
Assignment 3.py
IP > AppData > Local > Packages > 5319275A.WhatsAppDesktop_cv1g1gvanyjgm > LocalState > sessions > AC8C44A748FF07C37CCCAA67D34A64146085A2BE > transfers > 2026-37 > email_finder.py
- Personal note from Rahul (rahul_23@gmail.com) sent yesterday.
- Invalid mentions: not-an-email, @missing-local.com, plain.text@
- Newsletter sign-up: newsletter+promo@my-site.org
"""
print("Original text:")
print(sample_text)
found = find_emails(sample_text)
print(f"Found {len(found)} email address(es) in the text:")
for email in found:
print(f" - {email}")
print("\nValidating individual strings with is_valid_email():")
test_cases = [
"john.doe@example.com",
"invalid-email",
"user@site",
"user@site.com",
"plain.text@",
"a.b-c_d+e@sub.domain.co.in",
]
for candidate in test_cases:
result = "VALID" if is_valid_email(candidate) else "INVALID"
print(f" {candidate:30s} -> {result}")
Launchpad
Ln 1, Col 1 Spaces: 4 UTF-8 LF { } Python Sign In Python 3.14.3 Go Live
25°C Sunny
Search
ENG IN
09:55 15-09-2026


File Edit Selection View Go Run Terminal Help
Search
Update
Explorer
Open Editors 2 unsaved
email_finder.py C:\Users\HP\AppData\... 7
Assignment 1.py C:\Users\HP\AppData\Lo...
Assignment 3.py C:\Users\HP\AppData\Lo...
No Folder Opened
You have not yet opened a folder.
Open Folder
Opening a folder will close all currently open editors. To keep them open, add a folder instead.
You can clone a repository locally.
Clone Repository
To learn more about how to use Git and source control in VS Code read our docs.
Outline
Timeline
email_finder.py 7
Assignment 1.py
Assignment 3.py
IP > AppData > Local > Packages > 5319275A.WhatsAppDesktop_cv1g1gvanyjgm > LocalState > sessions > AC8C44A748FF07C37CCCAA67D34A64146085A2BE > transfers > 2026-37 > email_finder.py
1 """
Problems 7 Output Debug Console Terminal Ports
PS C:\Users\HP> python -u "c:\Users\HP\AppData\Local\Packages\5319275A.WhatsAppDesktop_cv1g1gvanyjgm\LocalState\sessions\AC8C44A748FF07C37CCCAA67D34A64146085A2BE\transfers\2026-37\email_finder.py"
Original text:
Please contact us for support:
- General queries: support@examplecorp.com
- Sales team: sales.team@business-hub.co.in
- Personal note from Rahul (rahul_23@gmail.com) sent yesterday.
- Invalid mentions: not-an-email, @missing-local.com, plain.text@
- Newsletter sign-up: newsletter+promo@my-site.org
Found 4 email address(es) in the text:
- support@examplecorp.com
- sales.team@business-hub.co.in
- rahul_23@gmail.com
- newsletter+promo@my-site.org
Validating individual strings with is_valid_email():
john.doe@example.com -> VALID
invalid-email -> INVALID
user@site -> INVALID
user@site.com -> VALID
plain.text@ -> INVALID
a.b-c_d+e@sub.domain.co.in -> VALID
PS C:\Users\HP>
powershell
Code
Launchpad
Ln 1, Col 1 Spaces: 4 UTF-8 LF { } Python Sign In Python 3.14.3 Go Live
25°C Sunny
Search
ENG IN
09:55 15-09-2026